

NumPy for Beginners — Study Guide

1. What is NumPy and Why Use It?

NumPy (**N**umerical **P**ython) gives Python fast, memory-efficient arrays and a huge set of mathematical functions to work with them.

Why not just use Python lists? - NumPy arrays are much faster for numerical work (they're implemented in C under the hood). - NumPy lets you do math on whole arrays at once, without writing loops. - Almost every data science / machine learning library (pandas, scikit-learn, TensorFlow, PyTorch) is built on top of NumPy.

```
import numpy as np    # the standard convention – always import it as np
```

2. Creating Arrays

The core object in NumPy is the **ndarray** (n-dimensional array).

```
import numpy as np

# From a List
a = np.array([1, 2, 3, 4])
print(a)           # [1 2 3 4]
print(type(a))     # <class 'numpy.ndarray'>
```

```
# 2D array (matrix) from a list of lists
b = np.array([[1, 2, 3], [4, 5, 6]])
print(b)
# [[1 2 3]
# [4 5 6]]
```

Handy array-creation functions

```
np.zeros((2, 3))      # 2x3 array of zeros
np.ones((3, 3))       # 3x3 array of ones
np.full((2, 2), 7)    # 2x2 array filled with 7
np.eye(3)             # 3x3 identity matrix
np.arange(0, 10, 2)   # [0 2 4 6 8] (like Python's range, but returns an array)
np.linspace(0, 1, 5)  # 5 evenly spaced numbers from 0 to 1
np.random.rand(2, 2)  # 2x2 array of random floats between 0 and 1
```

Checking array properties

```
a = np.array([[1, 2, 3], [4, 5, 6]])

a.shape    # (2, 3) -> 2 rows, 3 columns
```

```
a.ndim      # 2      -> number of dimensions
a.size      # 6      -> total number of elements
a.dtype     # dtype('int64') -> data type of elements
```

Practice: Section 2

1. Create a 1D array containing the numbers 10 to 50 (in steps of 10).
 2. Create a 4x4 identity matrix.
 3. Create a 3x3 array of all 9s using `np.full`.
 4. For the array `np.array([[1,2],[3,4],[5,6]])`, print its shape, ndim, and size.
-

3. Indexing and Slicing

Works similarly to Python lists, but extends naturally to multiple dimensions.

```
a = np.array([10, 20, 30, 40, 50])
```

```
a[0]          # 10
a[-1]         # 50
a[1:4]        # [20 30 40]
a[:3]         # [10 20 30]
a[::2]        # [10 30 50] -> every 2nd element
```

2D indexing

```
b = np.array([[1, 2, 3],
               [4, 5, 6],
               [7, 8, 9]])
```

```
b[0, 0]       # 1      -> row 0, col 0
b[1, :]       # [4 5 6] -> entire row 1
b[:, 2]       # [3 6 9] -> entire column 2
b[0:2, 0:2]   # [[1 2],[4 5]] -> top-left 2x2 block
```

Boolean (conditional) indexing — very commonly used

```
a = np.array([1, 2, 3, 4, 5, 6])
```

```
a[a > 3]      # [4 5 6] -> elements greater than 3
a[a % 2 == 0] # [2 4 6] -> even elements
```

```
a[a > 3] = 0   # replace all elements > 3 with 0
```

Practice: Section 3

1. Given `a = np.array([5, 10, 15, 20, 25, 30])`, get every element from index 1 to 4 (inclusive of 1, exclusive of 5... wait, get elements at positions 1 through 4).
2. Given `b = np.array([[1,2,3],[4,5,6],[7,8,9]])`, extract the middle column.
3. Given `a = np.array([2, 7, 3, 9, 4, 11])`, extract only the values greater than 5.

4. Replace all negative numbers in `np.array([1, -2, 3, -4, 5])` with 0.
-

4. Array Operations (Vectorization)

The big superpower of NumPy: apply an operation to an entire array at once — no loops needed.

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
```

```
a + b      # [5 7 9]
a - b      # [-3 -3 -3]
a * b      # [4 10 18]  -> element-wise multiplication
a / b      # [0.25 0.4 0.5]
a ** 2     # [1 4 9]
```

```
a + 10     # [11 12 13]  -> "broadcasting" a scalar to every element
```

Useful math functions

```
np.sqrt(a)    # square root of each element
np.exp(a)     # e^x for each element
np.log(a)     # natural log of each element
np.abs(-a)    # absolute value
```

Aggregation functions

```
a = np.array([[1, 2, 3], [4, 5, 6]])
```

```
a.sum()        # 21          -> sum of all elements
a.sum(axis=0)   # [5 7 9]     -> sum down each column
a.sum(axis=1)   # [6 15]      -> sum across each row
```

```
a.mean()       # 3.5
a.max()        # 6
a.min()        # 1
a.std()        # standard deviation
a.argmax()     # index of the max value (flattened)
```

Tip on axis: `axis=0` moves *down* the rows (collapses rows, one result per column). `axis=1` moves *across* columns (collapses columns, one result per row).

Practice: Section 4

1. Given `a = np.array([1, 2, 3, 4])`, compute the square of every element.
 2. Given `a = np.array([2, 4, 6])` and `b = np.array([1, 2, 3])`, compute `a / b`.
 3. Given `m = np.array([[1,2],[3,4],[5,6]])`, find the sum of each column and the sum of each row.
 4. Find the mean and standard deviation of `np.array([4, 8, 6, 5, 3, 9])`.
-

5. Reshaping Arrays

```
a = np.arange(12)           # [0 1 2 3 4 5 6 7 8 9 10 11]

a.reshape(3, 4)             # reshape into 3 rows, 4 columns
a.reshape(4, 3)             # reshape into 4 rows, 3 columns
a.reshape(2, -1)            # -1 means "figure this dimension out
                             automatically"

a.flatten()                 # collapse back to 1D
a.T                         # transpose (flip rows/columns) for 2D arrays
```

Common gotcha: reshape only works if the total number of elements stays the same. `np.arange(12).reshape(3, 5)` will throw an error, because $3 \times 5 = 15 \neq 12$.

Practice: Section 5

1. Create `np.arange(1, 17)` and reshape it into a 4x4 matrix.
 2. Reshape the same array into shape `(2, -1)` — what shape does it end up as?
 3. Take a 2x3 array and transpose it. What's the new shape?
-

6. Broadcasting

Broadcasting lets NumPy perform operations on arrays of *different* shapes, by automatically “stretching” the smaller one — without actually copying data.

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])      # shape (2, 3)

b = np.array([10, 20, 30])    # shape (3,)

a + b
# [[11 22 33]
#  [14 25 36]]
# b was "broadcast" across each row of a
```

Rule of thumb: shapes are compatible if, comparing dimensions from the right, they're equal or one of them is 1.

Practice: Section 6

1. Given `a = np.array([[1,2,3],[4,5,6],[7,8,9]])` and `b = np.array([1,0,1])`, what does `a + b` produce?
 2. Given a column vector `c = np.array([[1],[2],[3]])` (shape `(3,1)`) and `d = np.array([10, 20, 30])` (shape `(3,)`), what does `c + d` produce? (Hint: this is a classic broadcasting example that produces a 3x3 grid.)
-

7. Useful Extras

```
np.concatenate([a, b])      # join two arrays
np.vstack([a, b])           # stack arrays vertically (row-wise)
np.hstack([a, b])           # stack arrays horizontally (column-wise)
np.sort(a)                  # sorted copy of the array
np.unique(a)                # unique values, sorted
np.where(a > 3, 1, 0)       # 1 where condition true, 0 otherwise

a.copy()                   # important! slicing gives a *view*, not a
copy -                     # modifying a slice modifies the original
                             array too
```

The #1 beginner trap: `b = a[0:3]` does NOT copy `a` — it's a *view*. Changing `b` changes `a`. Use `b = a[0:3].copy()` if you need an independent copy.

8. Quick Reference Cheat Sheet

Task	Code
Create array	<code>np.array([1,2,3])</code>
Array of zeros	<code>np.zeros((r,c))</code>
Range of numbers	<code>np.arange(start, stop, step)</code>
Evenly spaced numbers	<code>np.linspace(start, stop, n)</code>
Shape / dims / size	<code>a.shape, a.ndim, a.size</code>
Reshape	<code>a.reshape(r, c)</code>
Transpose	<code>a.T</code>
Element-wise math	<code>a + b, a * b, a ** 2</code>
Sum / mean / max	<code>a.sum(), a.mean(), a.max()</code>
Sum by axis	<code>a.sum(axis=0) (cols), a.sum(axis=1) (rows)</code>
Filter with condition	<code>a[a > 5]</code>
Independent copy	<code>a.copy()</code>

9. Final Mixed Practice

Try these without looking back at the notes — they combine multiple concepts.

1. Create a 5x5 array of random integers between 1 and 100 (hint: `np.random.randint(low, high, size)`), then find the max value in each row.
2. Create an array of the numbers 1–20, reshape it into a 4x5 matrix, and replace every even number with -1.
3. Given two arrays `prices = np.array([100, 200, 150, 300])` and `discount = 0.1`, compute the discounted prices in one line using broadcasting.

